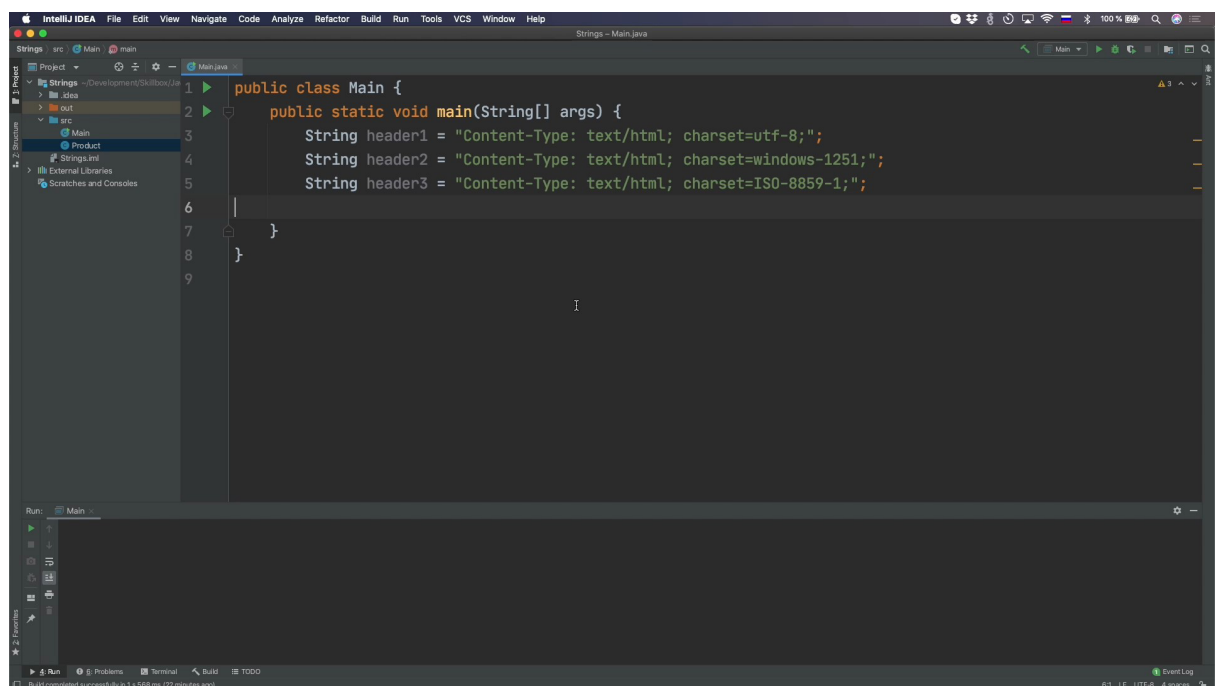


9.5 Методы работы с подстроками

00:04–00:17 Интро

Привет! В этом материале мы познакомимся с методами, позволяющими работать с фрагментами строк — подстроками: находить их позицию, извлекать их, объединять, а также форматировать строки определённым образом.

00:17–07:32 Методы определения подстрок



Представьте, что у вас есть строки подобного вида и вам нужно создать метод, который будет вычленять из таких строк названия кодировок: UTF-8, Windows-1251, ISO-8859-1 и так далее. Как это можно сделать?

Можно сначала воспользоваться специальными методами, которые нам позволят определить позицию подстрок — “charset=” и точка с запятой, следующие после “charset=”, и затем вычленить ту

подстроку, которая находится между этими двумя подстроками. Давайте по шагам применим эти методы.

Для этого создадим метод, который будет возвращать кодировку и в который в качестве параметра мы будем передавать наши строки:

```
public static String getEncoding (String header)
```

Теперь определим позицию, после которой следует строка с кодировкой:

```
String charsetStr = "charset=";
```

Воспользуемся специальным методом `indexOf`, определяющим позицию в строке той подстроки, которая будет передана в этот метод в качестве параметра.

```
int start = header.indexOf(charsetStr);
```

Поскольку сейчас нас интересует стартовая позиция не строки `charset`, а подстроки с именем кодировки, то сюда имеет смысл добавить длину строки:

```
int start = header.indexOf(charsetStr) + charsetStr.length();
```

Это как раз и будет позиция первого символа имени нашей кодировки.

Теперь нас будет интересовать позиция символа «;». Добавляем ещё одну переменную, указывая в качестве второго параметра отступ — это номер символа, после которого необходимо искать «;». В данном случае это номер символа `start`.

```
int end = header.indexOf(";", start );
```

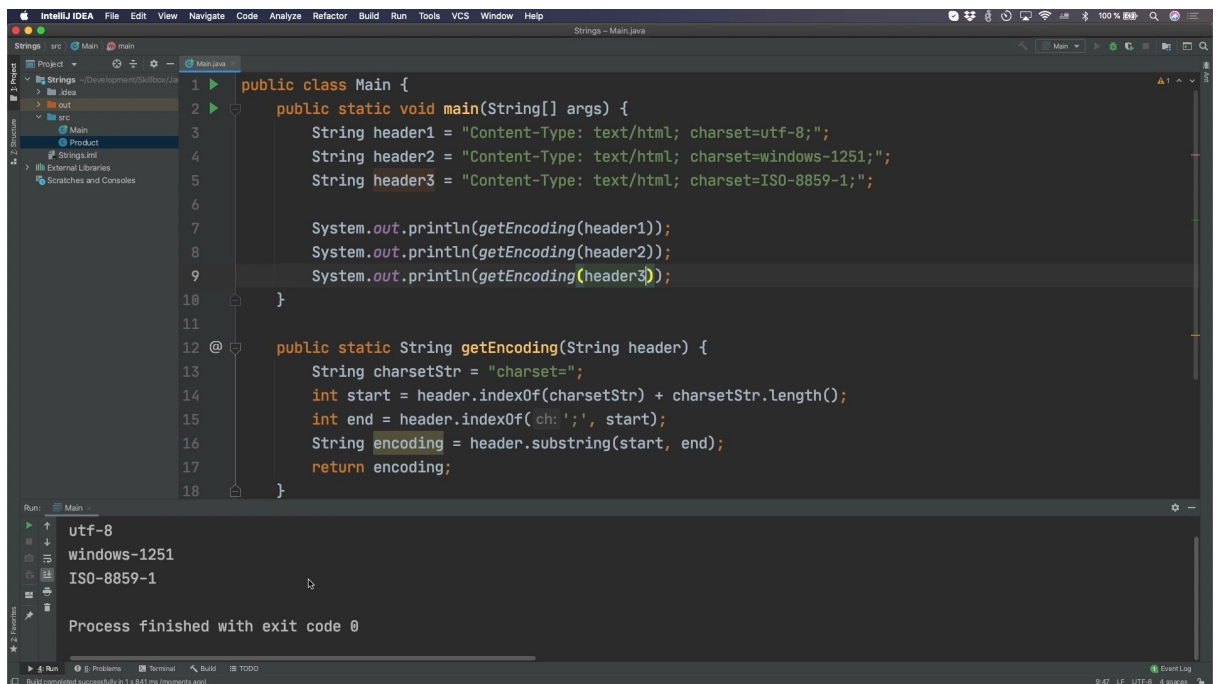
Затем получим подстроку

```
String encoding = header.substring(start, end);
```

и вернём эту переменную в качестве результата:

```
return encoding;
```

Выводим в консоль и видим, что всё работает правильно:

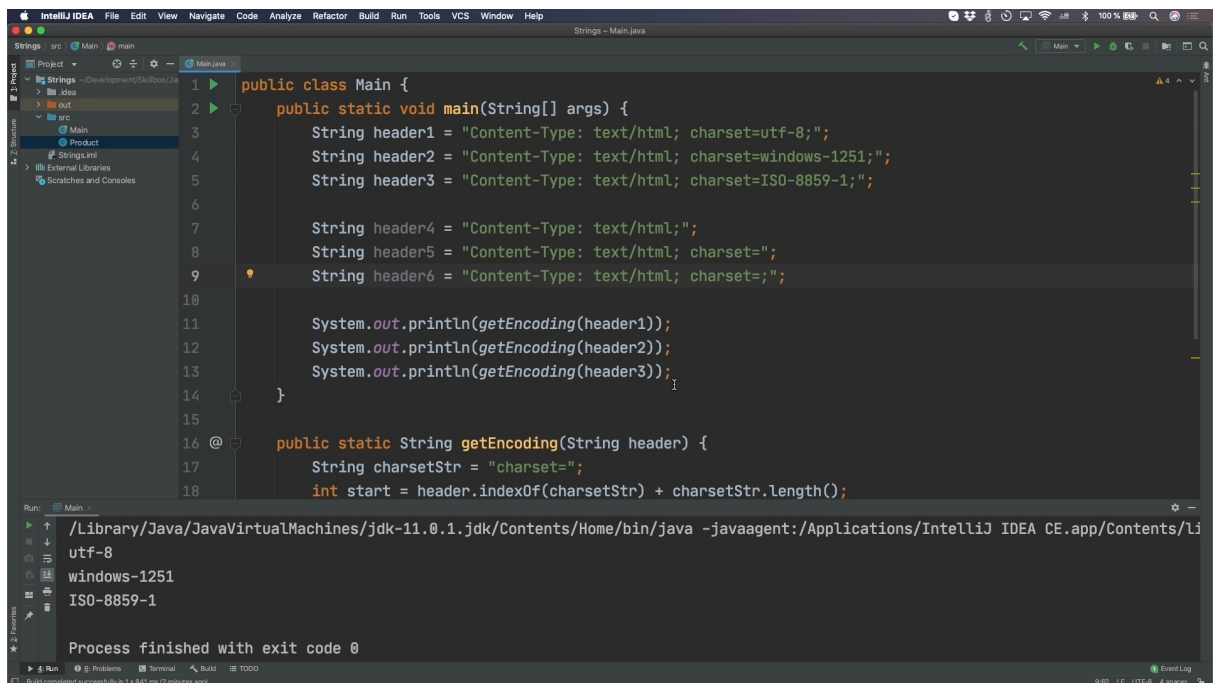


The screenshot shows the IntelliJ IDEA IDE with a Java file named 'Main.java'. The code defines a 'Main' class with a 'main' method and a static 'getEncoding' method. The 'main' method calls 'getEncoding' on three different headers. The 'getEncoding' method finds the start and end indices of the charset parameter in a header string and returns the substring between them. Below the code editor, the 'Run' tab shows the output of the program: 'utf-8', 'windows-1251', and 'ISO-8859-1', followed by 'Process finished with exit code 0'.

```
public class Main {  
    public static void main(String[] args) {  
        String header1 = "Content-Type: text/html; charset=utf-8;";  
        String header2 = "Content-Type: text/html; charset=windows-1251;";  
        String header3 = "Content-Type: text/html; charset=ISO-8859-1;";  
  
        System.out.println(getEncoding(header1));  
        System.out.println(getEncoding(header2));  
        System.out.println(getEncoding(header3));  
    }  
  
    public static String getEncoding(String header) {  
        String charsetStr = "charset=";  
        int start = header.indexOf(charsetStr) + charsetStr.length();  
        int end = header.indexOf(';', start);  
        String encoding = header.substring(start, end);  
        return encoding;  
    }  
}
```

Run: Main
utf-8
windows-1251
ISO-8859-1
Process finished with exit code 0

Обратите ещё раз внимание и запомните, что метод `substring` в качестве второго параметра принимает номер символа, до которого нужно извлечь подстроку, не включая символ `end`.



```
1 public class Main {
2     public static void main(String[] args) {
3         String header1 = "Content-Type: text/html; charset=utf-8;";
4         String header2 = "Content-Type: text/html; charset=windows-1251;";
5         String header3 = "Content-Type: text/html; charset=ISO-8859-1;";
6
7         String header4 = "Content-Type: text/html;";
8         String header5 = "Content-Type: text/html; charset=";
9         String header6 = "Content-Type: text/html; charset=";
10
11         System.out.println(getEncoding(header1));
12         System.out.println(getEncoding(header2));
13         System.out.println(getEncoding(header3));
14     }
15
16     public static String getEncoding(String header) {
17         String charsetStr = "charset=";
18         int start = header.indexOf(charsetStr) + charsetStr.length();
19
20         System.out.println(header);
21         System.out.println(charsetStr);
22         System.out.println(start);
23         System.out.println(header.substring(start));
24     }
25 }
```

Run: Main

/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li

utf-8

windows-1251

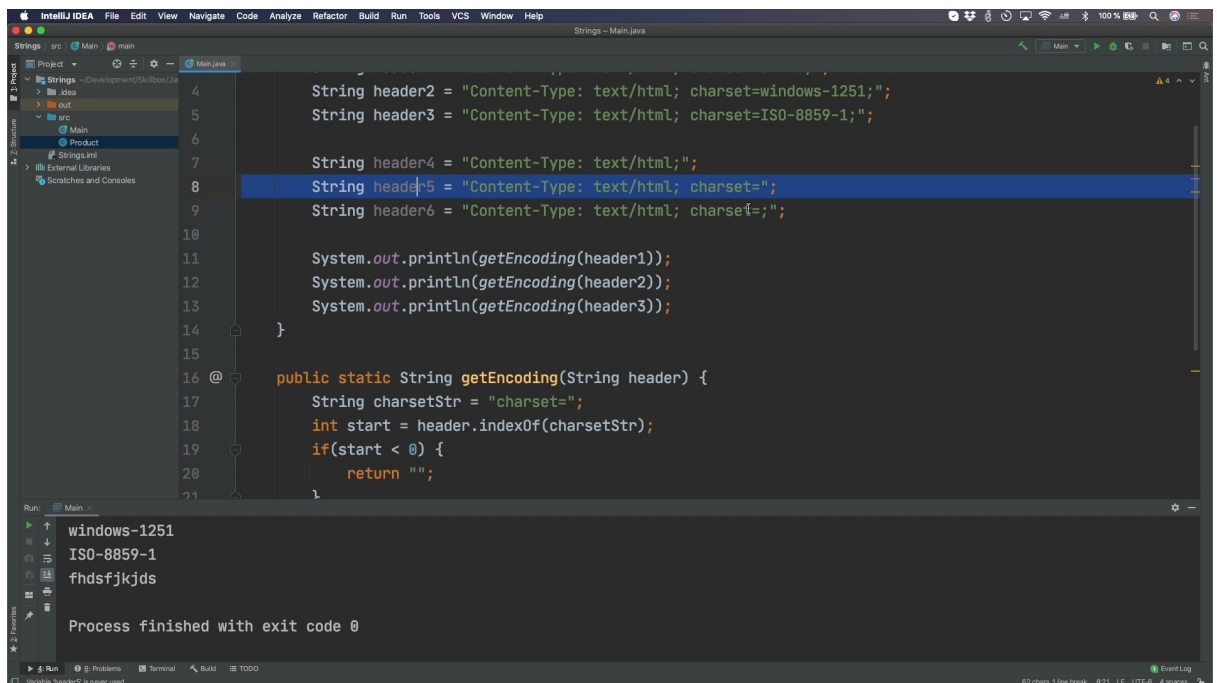
ISO-8859-1

Process finished with exit code 0

Теперь представьте, что в метод попадут вот такие строки (см. скрин выше) без указания кодировки. В первой строке нет слова `charset`; во второй оно есть, но только со знаком равно; в третьей есть и `charset` со знаком равно, и точка с запятой. Очевидно, метод `GetEncoding` в таком случае работать не будет.

Давайте сделаем так, чтобы метод работал правильно. В первом случае подстрока `charset` не будет найдена, и метод `indexOf` вернёт значение, равное `(-1)`, что будет означать, что переданная ему в качестве параметра подстрока отсутствует в строке. Давайте, чтобы проверять значение переменной `start`, уберём из её строки значение длины и перенесём значение длины в строку `String encoding`.

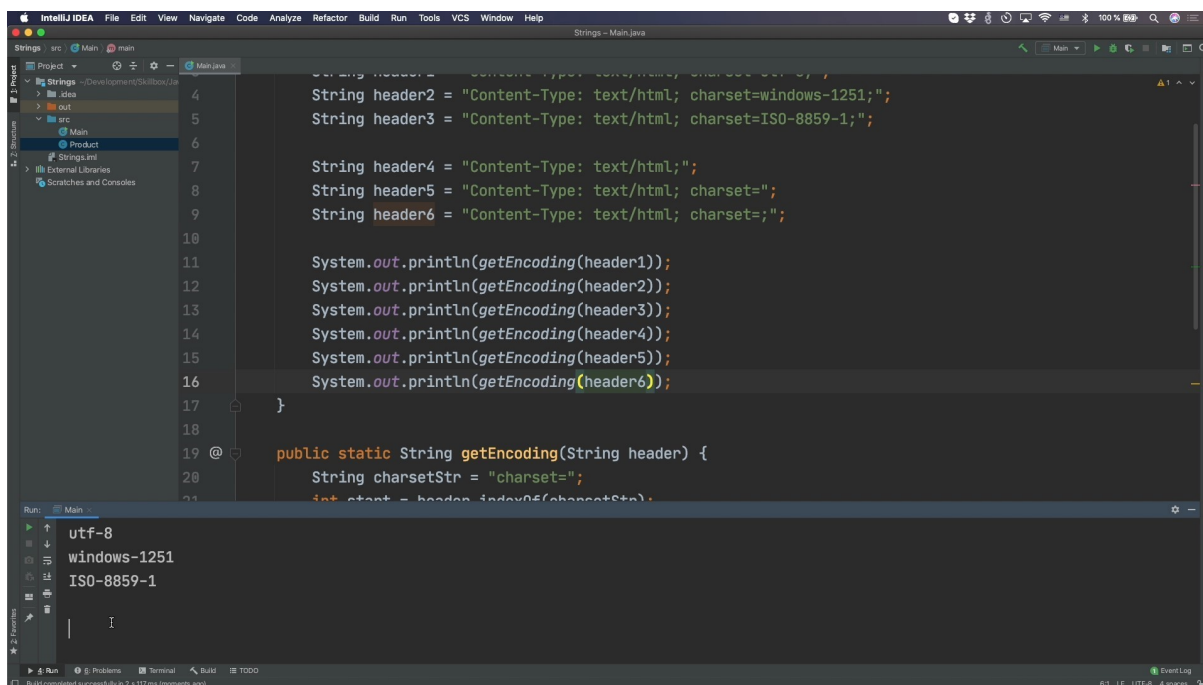
Добавляем проверку:



Во втором случае нет точки с запятой. Переменная `end` будет равна (-1). Чтобы это исправить, убираем строку `end` и добавляем в проверку условие `|| end < 0`.

В третьем случае обе подстроки будут найдены, и в существующем коде будет возвращена пустая строка.

Запускаем код и видим, что он работает верно: найдено три кодировки и три пустые строки.



Таким образом, мы можем работать со строками, используя различные методы поиска и вычленения подстрок.

Первый метод — `indexOf`, который позволяет найти подстроку в строке, её позицию, и если подстрока не найдена — метод возвращает значение (-1).

Второй метод — `indexOf` со вторым параметром, который позволяет искать подстроку после определённого символа.

Аналогичные методу `indexOf` методы `lastIndexOf` с одним и двумя параметрами, которые ищут не с начала, а с конца строки.

Также метод `substring` с одним параметром, который позволяет получить подстроку, начиная с определённого символа до конца строки; и с двумя параметрами, которые позволяют получить подстроку от и до какого-то символа.

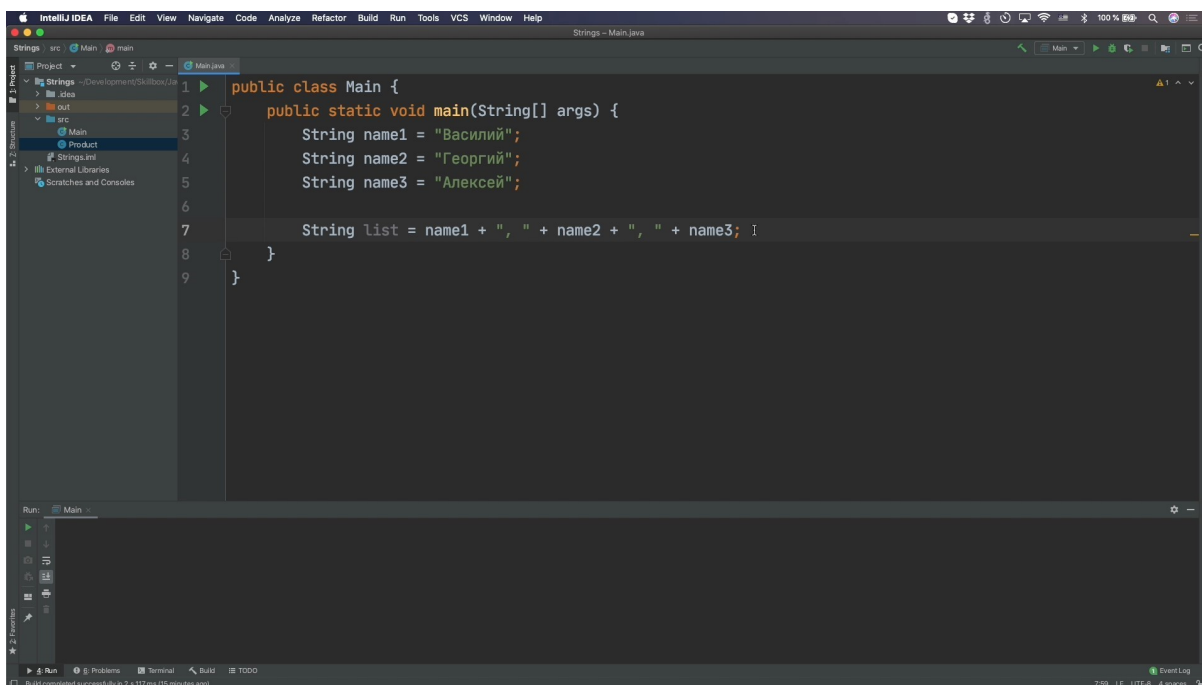
Методы работы с подстроками

```
.indexOf(".txt");  
.indexOf(".txt", from);  
.lastIndexOf(".zip");  
.lastIndexOf(".zip", from);  
.substring(from);  
.substring(from, to);
```

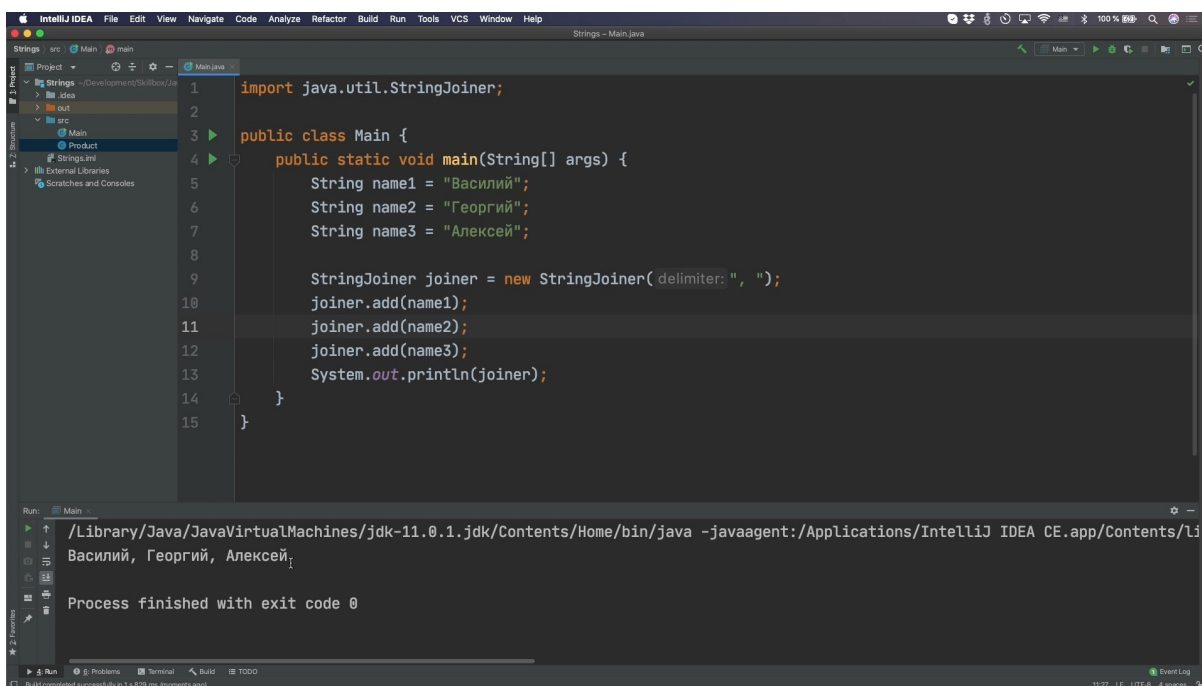
07:32–10:28 Объединение строк

Давайте теперь познакомимся с методами, позволяющими объединять строки. Часто бывает необходимо объединить несколько строк, используя разделитель, символ или строку. Например, у нас есть список имён и его нужно объединить в общую строку, в которой эти имена будут следовать через запятую.

Конечно, можно сделать так:



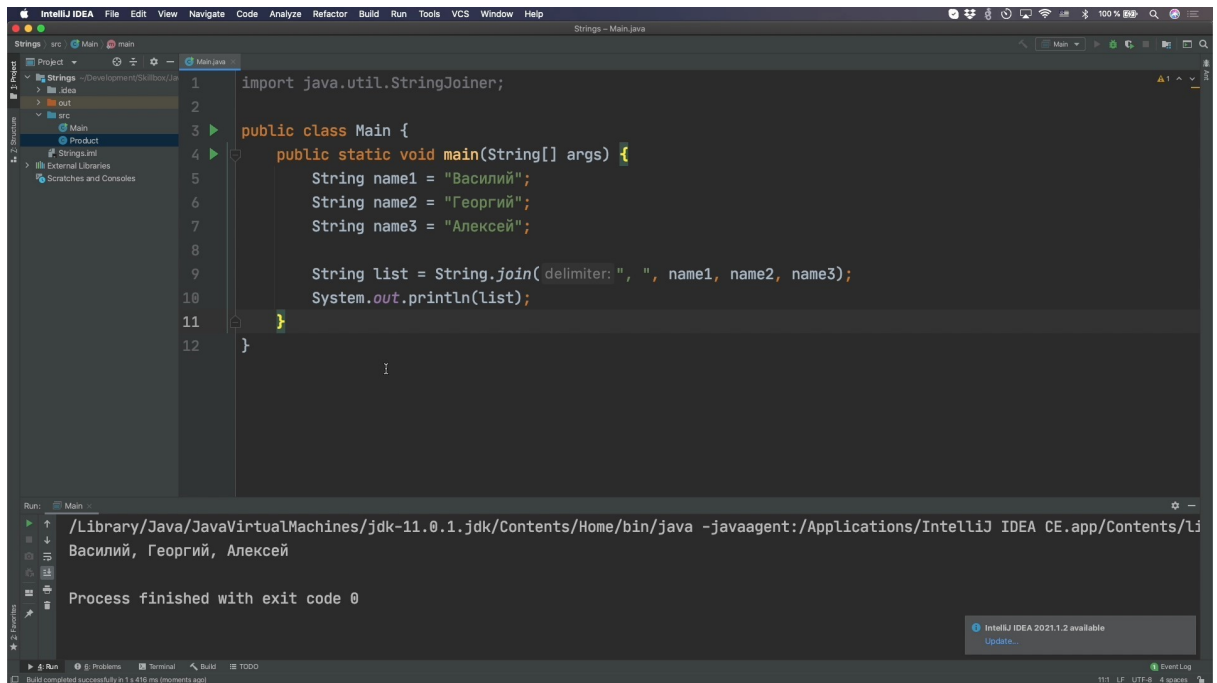
Но такая запись не совсем удобна. Если, к примеру, у вас не три имени, а 50, то будет уже очень большая и некрасивая строка, сложно поддерживаемая. Поэтому разработчики Java создали специальный класс, который называется `StringJoiner`. Он позволяет объединять строки, используя разделитель, который передаётся в конструктор этого класса.



Мы получаем ровно тот результат, который и хотели. Чем хорош этот класс? Он хорош тем, что если вы не будете больше добавлять

никакие строки, то разделитель тоже добавляться не будет. То есть он добавляется только между строками.

Так же и у класса `String` есть статический метод `join`, который позволяет объединять через разделитель передаваемые подстроки.



```
1 import java.util.StringJoiner;
2
3 public class Main {
4     public static void main(String[] args) {
5         String name1 = "Василий";
6         String name2 = "Георгий";
7         String name3 = "Алексей";
8
9         String list = String.join(" ", name1, name2, name3);
10        System.out.println(list);
11    }
12 }
```

Run: Main

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
Василий, Георгий, Алексей

Process finished with exit code 0
```

Таким образом, для объединения строк через разделитель можно воспользоваться как специальным классом `StringJoiner`, так и статическим методом `join` класса `String`, в который в качестве первого параметра передаётся разделитель, а в качестве второго и последующих параметров — те строки, которые необходимо объединить с использованием этого разделителя.

Объединение строк

```
StringJoiner joiner =  
    new StringJoiner(", ");  
joiner.add(name1);  
joiner.add(name2);  
joiner.add(name3);  
System.out.println(joiner);
```

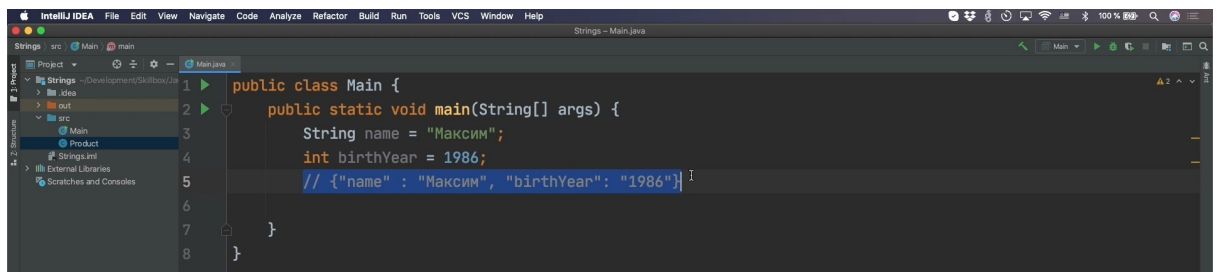
Объединение строк

```
String result = String.join(  
    ", ", name1, name2, name3  
);
```

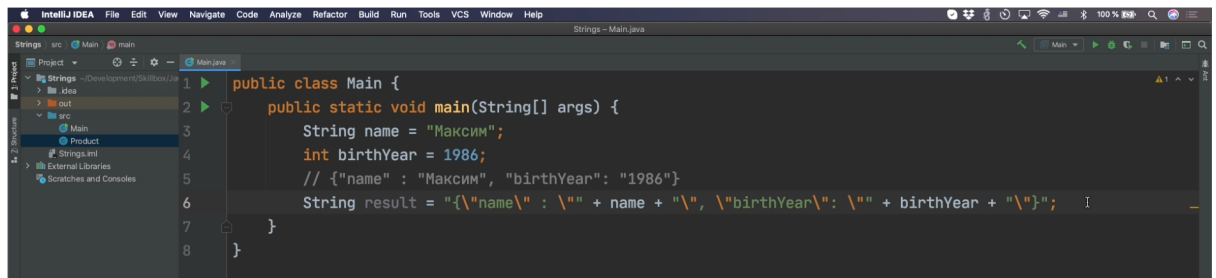
10:28–14:23 Форматирование строк

Ещё одна полезная возможность, которая делает код более удобным и кратким, — это возможность форматирования строк.

Представьте, что у вас есть две строки и вам нужно вывести эти две переменные в таком формате, как указано на скриншоте:

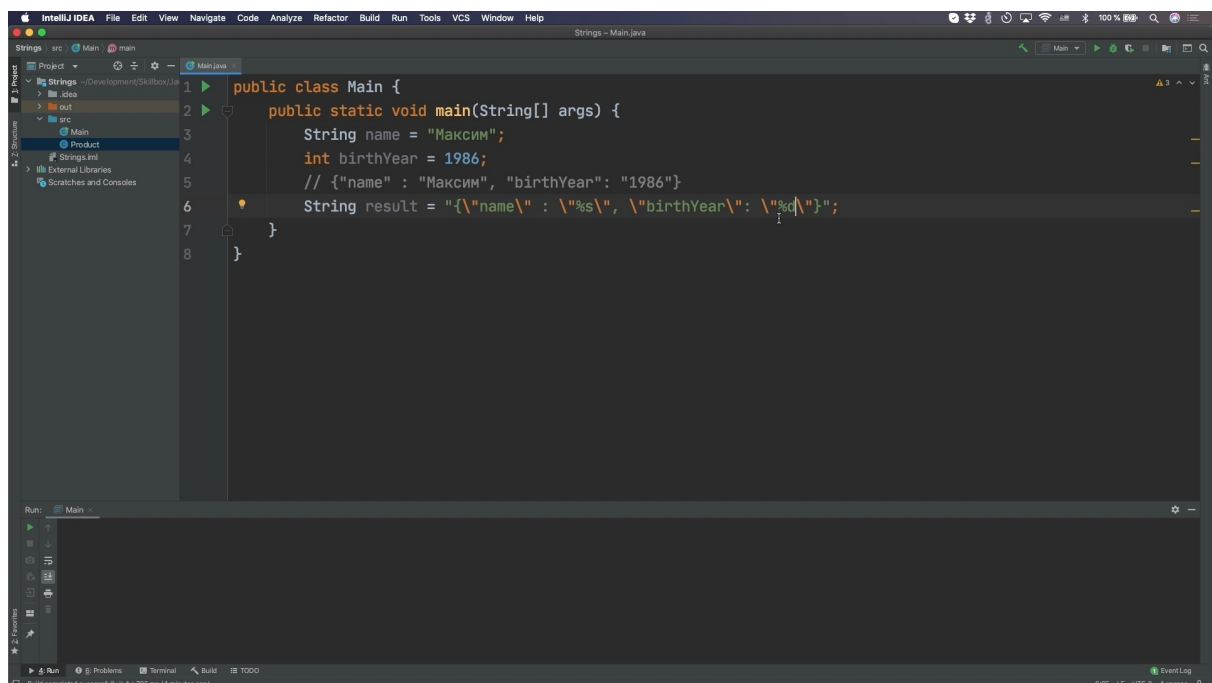


Мы можем, конечно, сделать следующим способом:



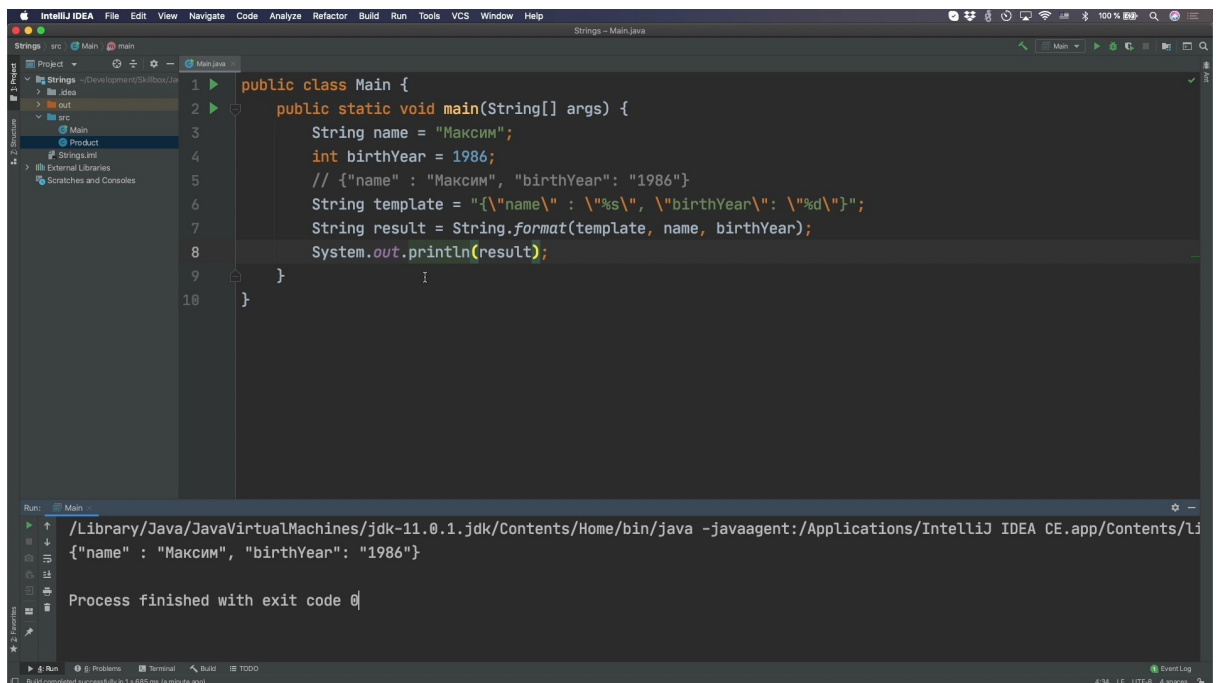
Такой способ будет не очень красивым и не очень поддерживаемым. В первую очередь, потому что этих переменных может быть много, и из-за этого строка может сильно разрастись. Во-вторых, потому что вместо переменной здесь может быть объект, у которого вызываются какие-то методы для получения этих данных — тогда строка будет ещё больше.

Поэтому, чтобы сделать такой код более удобным и поддерживаемым, разработчики придумали механизм форматирования строк. Здесь вместо этих переменных мы можем написать специальные символы:



s означает строку, а d — целое число. По сути, мы создали шаблон.

Чтобы им воспользоваться, пишем:



```
1 public class Main {
2     public static void main(String[] args) {
3         String name = "Максим";
4         int birthYear = 1986;
5         // {"name" : "Максим", "birthYear": "1986"}
6         String template = "{\\\"name\\\" : \\\"%s\\\", \\\"birthYear\\\" : \\\"%d\\\"}";
7         String result = String.format(template, name, birthYear);
8         System.out.println(result);
9     }
10 }
```

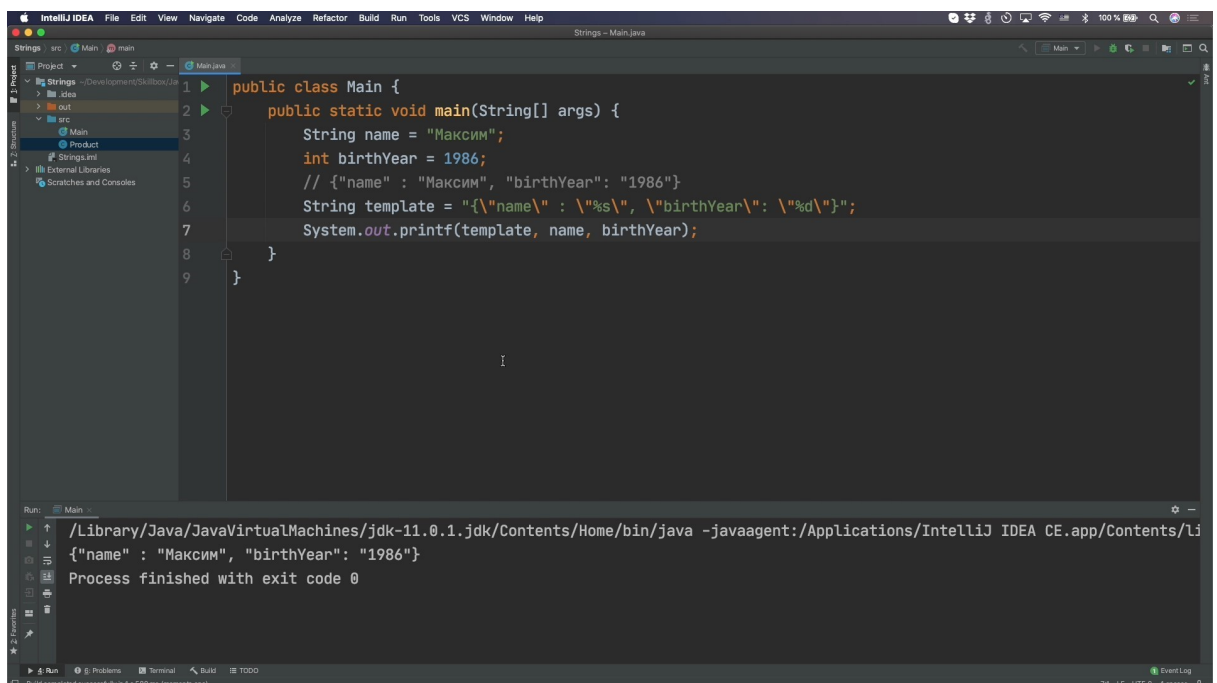
Run: Main

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
{"name" : "Максим", "birthYear": "1986"}

Process finished with exit code 0
```

При выводе в консоль видим, что всё работает верно.

У `System.out.println` есть вариант `System.out.printf`, который позволяет форматировать строки и сразу выводить их в консоль.



```
1 public class Main {
2     public static void main(String[] args) {
3         String name = "Максим";
4         int birthYear = 1986;
5         // {"name" : "Максим", "birthYear": "1986"}
6         String template = "{\\\"name\\\" : \\\"%s\\\", \\\"birthYear\\\" : \\\"%d\\\"}";
7         System.out.printf(template, name, birthYear);
8     }
9 }
```

Run: Main

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
{"name" : "Максим", "birthYear": "1986"}

Process finished with exit code 0
```

Специальных символов, которые можно подставлять в шаблоны, существует довольно много. Основные из них вы можете увидеть на слайде:

Форматирование строк

Обозначение	Описание
%s	строка (String)
%d	целое число
%f	число float или double
%b	boolean
%n	перенос строки
%t	дата (Date)
%%	символ процента

Обратите внимание на символ переноса строки %n. Вы помните, что в разных операционных системах это разные символы. Так вот, символ %n в шаблоне подставляет символ переноса строки, использующийся в операционной системе, на которой запускается ваша программа.

14:23–14:52 Итоги

Итак, вы познакомились с различными аспектами работы со строками: с поиском и извлечением подстрок, с объединением строк через заданный разделитель, а также с методами и шаблонами форматирования строк.

В следующем материале мы затронем тему, которую важно хорошо знать и понимать и вопросы по которой часто задают на собеседовании, — это так называемый string pool.